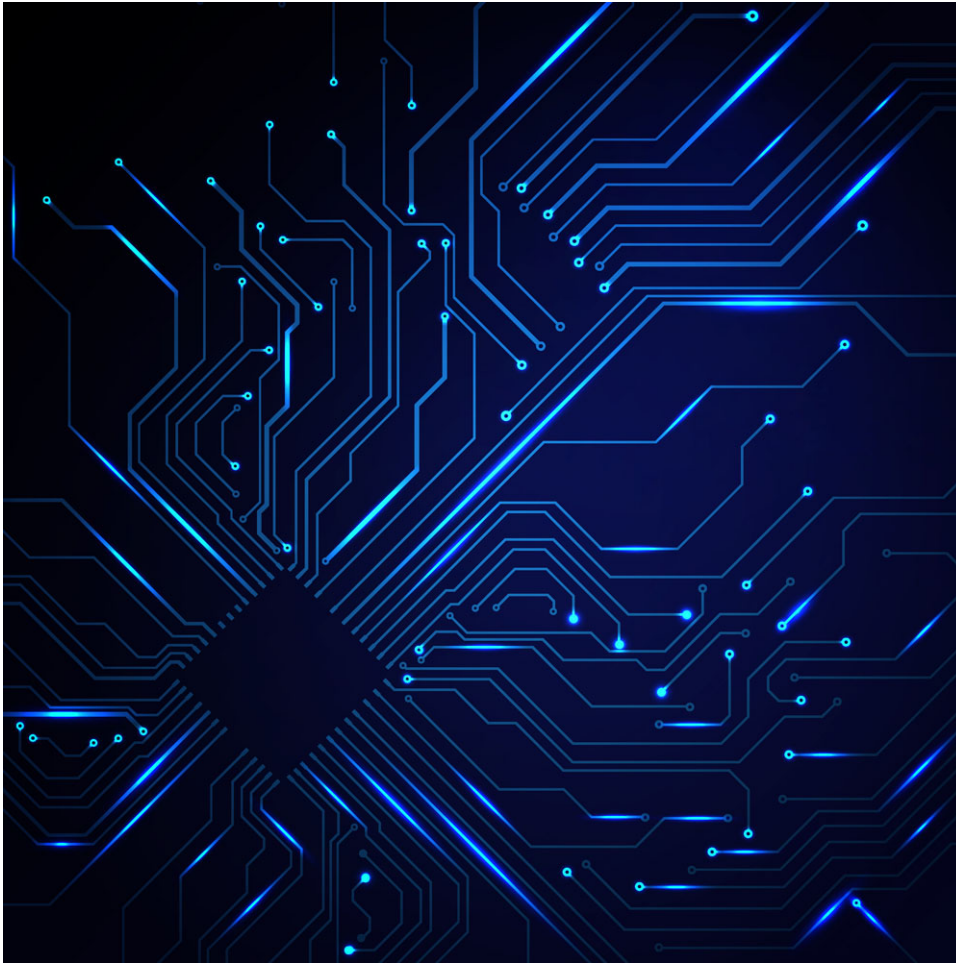




LAB10 TIMER

Name: 黃翊宏

Student ID: 01357101

Note

1. All files must be converted to PDF format before submission.
2. **Submission deadline: Next Tuesday at 1:00 PM**
3. Each person submits one assignment.

Main Code Screenshot

time.sv:

```

V timer.sv x
de0_empty > design > V timer.sv
1  module timer(
2      input clk,
3      input reset,
4      output logic [2:0] sec_tens, min_tens,
5      output logic [3:0] sec_units, min_units
6  );
7      // Hour Timer 00:00 ~ 59:59
8      logic sec_unit_carry, sec_ten_carry, min_unit_carry, min_ten_carry;
9      unit sec_unit(
10         .clk(clk), .reset(reset), .Cin(1'b1), .q(sec_units), .Carry(sec_unit_carry)
11     );
12
13     ten_digit sec_ten(
14         .clk(clk), .reset(reset), .Cin(sec_unit_carry), .q(sec_tens), .Carry(sec_ten_carry)
15     );
16
17     unit min_unit(
18         .clk(clk), .reset(reset), .Cin(sec_ten_carry), .q(min_units), .Carry(min_unit_carry)
19     );
20
21     ten_digit min_ten(
22         .clk(clk), .reset(reset), .Cin(min_unit_carry), .q(min_tens), .Carry(min_ten_carry)
23     );
24 endmodule

```

unit.sv

```

V timer.sv V unit.sv x
de0_empty > design > V unit.sv
1  module unit(
2      input clk,
3      input reset,
4      input Cin,
5      output logic [3:0] q,
6      output logic Carry
7  );
8      logic a, b, c, d, Da, Db, Dc, Dd;
9      dfflipflop dff1(
10         .d(Da), .clk(clk), .reset(reset), .q(a)
11     );
12
13     dfflipflop dff2(
14         .d(Db), .clk(clk), .reset(reset), .q(b)
15     );
16
17     dfflipflop dff3(
18         .d(Dc), .clk(clk), .reset(reset), .q(c)
19     );
20
21     dfflipflop dff4(
22         .d(Dd), .clk(clk), .reset(reset), .q(d)
23     );
24
25     assign Da = (~Cin & a) | (a & ~d) | (Cin & b & c & d);
26     assign Db = (~Cin & b) | (b & ~c) | (b & ~d) | (Cin & ~b & c & d);
27     assign Dc = (~Cin & c) | (c & ~d) | (Cin & ~a & ~c & d);
28     assign Dd = ((~Cin)&d) | (Cin&(~d));
29     assign Carry = Cin & a & d;
30     assign q[3:0] = {a,b,c,d};
31 endmodule

```

ten_digit.sv

```
timer.sv × unit.sv ten_digit.sv ×
de0_empty > design > ten_digit.sv
1  module ten_digit(
2      input clk,
3      input reset,
4      input Cin,
5      output [2:0]q,
6      output logic Carry
7  );
8      logic a, b, c, Da, Db, Dc;
9
10     dff dff1(
11         .d(Da), .clk(clk), .reset(reset), .q(a)
12     );
13
14     dff dff2(
15         .d(Db), .clk(clk), .reset(reset), .q(b)
16     );
17
18     dff dff3(
19         .d(Dc), .clk(clk), .reset(reset), .q(c)
20     );
21
22     assign Da = ((~Cin)&a) | (a&(~c)) | (Cin&c&b);
23     assign Db = ((~Cin)&b) | (b&(~c)) | (Cin&(~a)&(~b)&c);
24     assign Dc = ((~Cin)&c) | (Cin&(~c));
25     assign Carry = Cin & a & c;
26     assign q[2:0] = {a,b,c};
27 endmodule
```

dff.sv

```
timer.sv dff.sv × unit.sv ten_digit.sv
de0_empty > design > dff.sv
1  module dff(
2      input d,
3      input clk,
4      input reset,
5      output logic q,
6      output logic q_n
7  );
8      //正邊緣觸發 D型正反器
9      always_ff @(posedge clk)
10         begin
11             if (reset) begin
12                 q <= 1'b0;
13                 q_n <= 1'b1;
14             end
15             else begin
16                 q <= d;
17                 q_n <= ~d;
18             end
19         end
20 endmodule
```

Testbench Screenshot

```
timer.vsv  tb_timer.vsv  unit.vsv  ten_digit.vsv
de0_empty > simulation > tb > tb_timer.vsv
1  module tb_timer;
2      logic clk, reset;
3      logic [2:0] sec_tens, min_tens;
4      logic [3:0] sec_units, min_units;
5
6      timer u_timer(
7          .clk(clk), .reset(reset), .sec_tens(sec_tens), .min_tens(min_tens), .sec_units(sec_units), .min_units(min_units)
8      );
9
10     always #5 clk = ~clk;
11     initial begin
12         clk = 0; reset = 1;
13         #10 reset = 0;
14         #10000 $stop;
15     end
16 endmodule
```

mcu.sv Code Screenshot

```
timer.vsv  mcu.vsv  unit.vsv  ten_digit.vsv
de0_empty > design > mcu.vsv
42  logic clk;
43  logic [3:0] sec_units, min_units;
44  logic [2:0] sec_tens, min_tens;
45
46  freg_div_23 freg1(
47      .clk(CLK),
48      .reset(~BTN[0]),
49      .clk_out(clk)
50  );
51
52  timer timer1(
53      .clk(clk),
54      .reset(~BTN[0]),
55      .sec_units(sec_units),
56      .sec_tens(sec_tens),
57      .min_units(min_units),
58      .min_tens(min_tens),
59  );
60
61  //bouns
62  /*day_timer timer2(
63      .clk(clk),
64      .reset(~BTN[0]),
65      .sec_units(sec_units),
66      .sec_tens(sec_tens),
67      .min_units(min_units),
68      .min_tens(min_tens),
69  );*/
```

```
timer.vsv  mcu.vsv  unit.vsv  ten_digit.vsv
de0_empty > design > mcu.vsv
70
71  ssd ssd1(
72      .a(sec_units),
73      .seg(HEX0)
74  );
75
76  ssd ssd2(
77      .a(sec_tens),
78      .seg(HEX1)
79  );
80
81  ssd ssd3(
82      .a(min_units),
83      .seg(HEX2)
84  );
85
86  ssd ssd4(
87      .a(min_tens),
88      .seg(HEX3)
89  );
90
91  //*****//
92
93
94  endmodule
95
```

Waveform and Result Screenshot (The values in the waveform must be clearly visible.)





Lab Summary and Reflection:

Summary

During this laboratory session, we were tasked with designing a 00:00–59:59 timer using D flip-flops within the Quartus environment on an Altera DE0 development board. The design process began by modularizing the counter into units and tens digits for both seconds and minutes: a four-bit D-flip-flop counter for the units digit that resets after reaching nine to generate a carry, and a three-bit counter for the tens digit that resets after five and propagates its carry to the next module. Karnaugh-map techniques were applied to derive minimized next-state equations for each D input, and corresponding SystemVerilog modules—`unit_sv`, `ten_digit_sv`, and `dflipflop_sv`—were instantiated within a top-level `time_sv` file. A comprehensive testbench was developed to simulate the expected counting sequence, and once the waveform matched the theoretical output, the signals were mapped to the HEX displays on the DE0 board through proper pin assignments for on-board verification .

In the bonus exercise, the counter was extended to serve as a 24-hour clock spanning 00:00 to 23:59. This extension involved introducing `hour_units` and `hour_tens` registers with conditional reset logic that triggers when the count reaches 24:00, forcing the circuit back to 00:00. The architecture diagram was updated accordingly, and the necessary modifications were made in SystemVerilog to implement this reset condition. Successful operation was again confirmed through both simulation and on-board testing .

Reflection

This laboratory experience deepened my understanding of synchronous digital design by reinforcing the process of translating state diagrams and tables into implementable HDL code and highlighting the value of Karnaugh-map simplification for optimizing logic. Building and verifying a modular counter architecture in SystemVerilog enhanced my appreciation for code reuse and abstraction, while debugging the testbench and interpreting waveform outputs sharpened my troubleshooting skills. Finally, deploying the design on the DE0 FPGA and interfacing with seven-segment displays provided practical insight into hardware-software integration challenges, such as pin mapping and timing considerations, lessons that will be invaluable in future digital system projects.